DS1302底层书写

底层

我们这里直接使用十进制的底层，解决一切问题，就不介绍BCD码了，首先是DS1302的底层，我们需要看一下原理图，确定一下引脚



这个引脚需要自己用sbit定义一下，如下所示，并且官方的底层用到了NOP，所以需要引入一下intrins这个头文件

```
#include "intrins.h"
sbit SDA = P2 ^ 3;
sbit RST = P1 ^ 3;
sbit SCK = P1 ^ 7;
```

然后就是我们根据芯片手册来书写时间写入和读取的相关代码

Table 3. Register Address/Definition

RTC

READ	WRITE	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0	RANGE
81h	80h	CH	10 Seconds			Seconds				00–59
83h	82h		10 Minutes			Minutes				00–59
85h	84h	12/$\overline{24}$	0	10 $\overline{\text{AM}}$/PM	Hour	Hour				1–12/0–23
87h	86h	0	0	10 Date		Date				1–31
89h	88h	0	0	0	10 Month	Month				1–12
8Bh	8Ah	0	0	0	0	0	Day			1–7
8Dh	8Ch	10 Year				Year				00–99
8Fh	8Eh	WP	0	0	0	0	0	0	0	—
91h	90h	TCS	TCS	TCS	TCS	DS	DS	RS	RS	—

CLOCK BURST

BFh	BEh

RAM

C1h	C0h		00-FFh
C3h	C2h		00-FFh
C5h	C4h		00-FFh
. . .	. . .		. . .
FDh	FCh		00-FFh

一、关键配置位说明

1. **WP写保护位（地址0x8E）**

- 当WP位置1时，芯片禁止写入操作

- 必须先将其置0才能进行时间参数配置

2. **CH时钟停止位（地址0x80）**

- 位于秒寄存器的最高位（bit7）
- 置1时会停止时钟振荡器（常见于初始化配置）
- 置0时启动时钟（详细说明见常见问题章节）

二、时间参数写入流程

```c
void Set_Rtc(unsigned char *ucRtc)
{
  unsigned char i;
  Write_Ds1302_Byte(0x8e, 0x00); // 1. 关闭写保护
  Write_Ds1302_Byte(0x80, 0x80); // 2. 停止时钟振荡器
  for (i = 0; i < 3; i++)
    Write_Ds1302_Byte(0x84 - 2 * i, ucRtc[i] / 10 % 10 * 16 + ucRtc[i] % 10); // 3. 写入BCD数据
  Write_Ds1302_Byte(0x8e, 0x80); // 4. 重新启用写保护
}
```

操作步骤解析：

1. **BCD码转换公式**：`十进制数值 → 高4位(十位)*16 + 低4位(个位)`

- 示例：数值11 → `1*16 + 1 = 0x11`

2. **地址映射**：

- 秒：0x80 → 0x84（写入地址自动递增特性）
- 分：0x82
- 时：0x84

三、时间参数读取流程

```c
void Read_Rtc(unsigned char *ucRtc)
{
  unsigned char i;
  unsigned temp;
  EA = 0; // 关闭全局中断保证时序完整性
  for (i = 0; i < 3; i++)
  {
    temp = Read_Ds1302_Byte(0x85 - 2 * i); // 读取地址自动递增
    ucRtc[i] = temp / 16 * 10 + temp % 16; // BCD转十进制
  }
  EA = 1; // 恢复中断
}
```

关键实现细节：

- **BCD逆转换**：`高4位*10 + 低4位`
- **地址对齐**：
  - 秒：0x81（读取地址需+1）

- 分：0x83
- 时：0x85

四、重要注意事项

1. **中断处理**：
   - 读取时必须关闭全局中断（EA=0）
   - 防止时序被打断导致读取错误
2. **写保护机制**：
   - 任何写操作前后必须配置WP位
   - 遵循"关闭保护→操作→恢复保护"的原子操作原则
3. **时钟精度保障**：
   - 初始化时建议先停止时钟（CH=1）

五、典型应用场景

```
// 设置时间为12:34:56
unsigned char time[] = {12,34, 56};
Set_Rtc(time);

// 读取当前时间
unsigned char current_time[3];
Read_Rtc(current_time);
// current_time[0]=时, [1]=分, [2]=秒
```

DS1302考点

其实他没什么考点，目前考到的就那么几个

整点触发xx功能

整点，顾名思义就是分秒为0的时候，那么这个其实很简单，我们直接判断分秒是否为0就可以了

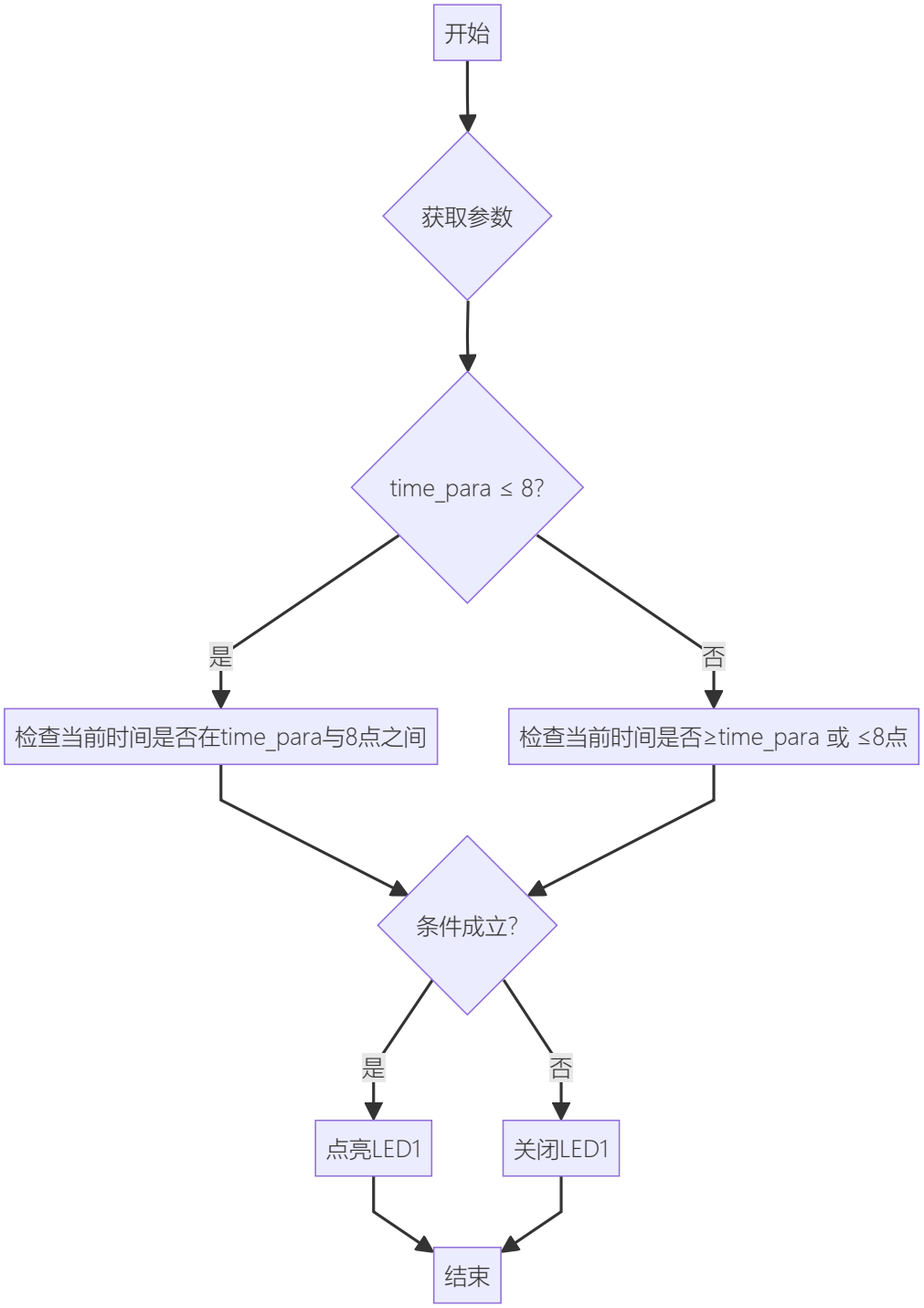
```
if(ucRtc[1]==0&&ucRtc[2]==0)
    //整点触发代码
```

时间段触发逻辑说明

当需要在设定时间段内触发LED点亮时，需考虑时间跨天情况：

1. **同一天情况（time_para ≤ 8时）**
   - 当前时间需同时满足：
     - 大于等于设定时间（time_para）
     - 小于等于固定时间点（8点）
2. **跨天情况（time_para > 8时）**
   - 当前时间满足以下任一条件：
     - 大于等于设定时间（time_para）且在当天23:59之前
     - 小于等于固定时间点（8点）且在次日00:00之后

通过分段判断实现24小时制的时间区间检测。



流程说明：

1. 通过判断设定时间与8点的大小关系确定是否跨天
2. 分别采用不同的区间判断策略：
   - 同天采用闭区间判断（AND逻辑）
   - 跨天采用分段判断（OR逻辑）
3. 最终根据时间匹配结果控制LED状态

该实现方式完整覆盖了24小时时间周期，可正确处理以下典型场景：

- 当time_para=6时，生效时段为06:00-08:00
- 当time_para=22时，生效时段为22:00-次日08:00
- 当time_para=8时，仅在8点整触发

```
if(time_para>8)
    if(ucRtc[0]>=time_para||ucRtc[0]<=8)
else
    if(ucRtc[0]>=time_para&&ucRtc[0]<=8)
```

闹钟

对于闹钟，他其实就是设置了一个另外的数据，我们采集后直接和他对比就行了

```
unsigned char alarm[3]={11,11,11}//11:11:11的闹钟
if(ucRtc[0]==alarm[0]&&ucRtc[1]==alarm[1]&&ucRtc[2]==alarm[2])
```

获取触发的时间

最近几届喜欢考，当我因为某个条件触发的时候，我将时间进行记录一下。

我们这里需要额外定义一个数组然后直接read就可以了

```
unsigned char tri_rtc[3];
if(条件触发)
    Read_Rtc(tri_rtc);
```

DS1302 常见问题

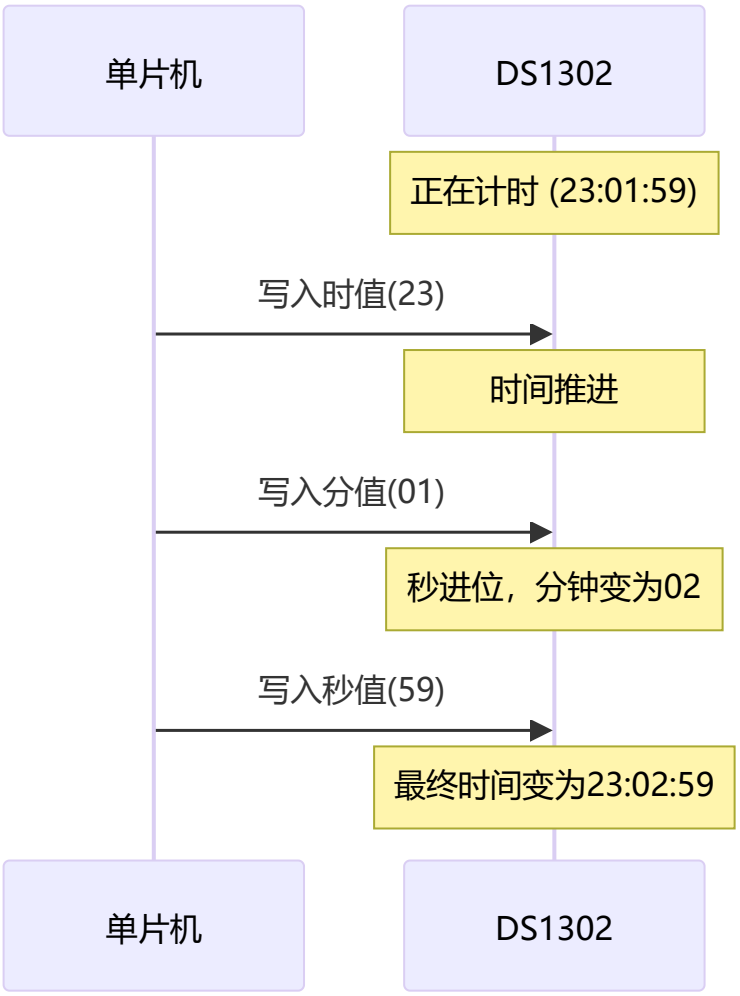
时间写入异常分析

问题描述

在向DS1302写入时间时，经常会遇到写入值与实际值不符的情况。例如，当我们尝试写入 `23:01:59` 时，实际读取值可能变成 `23:02:59`。

问题原因

这个问题的根本原因在于写入过程中DS1302仍在计时。以下时序图展示了问题发生的过程：



解决方案

在写入时间前，需要先停止DS1302的计时功能。这可以通过设置CH位（Clock Halt）为1来实现，使芯片进入低功耗模式。

```c
void Set_Rtc(unsigned char *ucRtc)
{
    unsigned char i;
    Write_Ds1302_Byte(0x8e, 0x00);     // 解除写保护
    Write_Ds1302_Byte(0x80, 0x80);     // 设置CH位，停止计时
    for (i = 0; i < 3; i++)
        Write_Ds1302_Byte(0x84 - 2 * i, ucRtc[i] / 10 % 10 *16 + ucRtc[i] % 10);
    Write_Ds1302_Byte(0x8e, 0x80);     // 重新开启写保护
}
```

时间读取优化

问题说明

DS1302使用SPI通信协议，对时序要求严格。在读取过程中，任何中断都可能导致通信错误。

优化方案

在读取过程中暂时关闭总中断，确保时序的完整性：

```
void Read_Rtc(unsigned char *ucRtc)
{
    unsigned char i;
    unsigned char temp;
    EA = 0;     // 关闭总中断，确保时序准确
    for (i = 0; i < 3; i++)
    {
        temp = Read_Ds1302_Byte(0x85 - 2 * i);
        ucRtc[i] = temp / 16 * 10 + temp % 16;     // BCD码转换为十进制
    }
    EA = 1;     // 恢复总中断
}
```